



512'5''1''2'
51A '5''1''A'

00000001 00000001

Binary Input/Output

Objectives

- ☐ To understand the differences between text and binary files
- ☐ To write programs that read, write, and/or append binary files
- ☐ To be able to detect and take appropriate action when file errors occur
- ☐ To be able to process files randomly
- ☐ To be able to create text files from binary files and vice versa
- ☐ To understand and be able to implement file merges
- ☐ To understand and be able to implement the classic sequential file update

13-1 Text versus Binary Streams

In this section, we compare and contrast text streams versus binary streams.

Topics discussed in this section:

Text and Binary Files

Differences between Text and Binary Files

State of a File

Opening Binary Files

Closing Binary Files

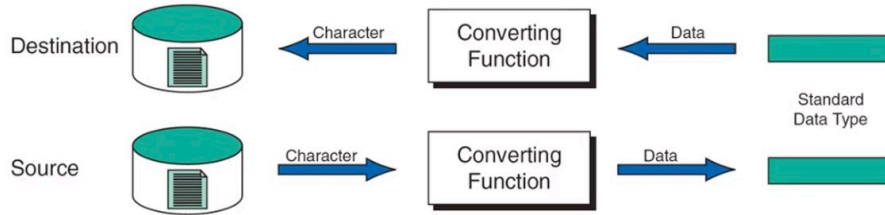


FIGURE 13-1 Reading and Writing Text Files

Note

Formatted input/output, character input/output, and string input/output functions can be used only with text files.

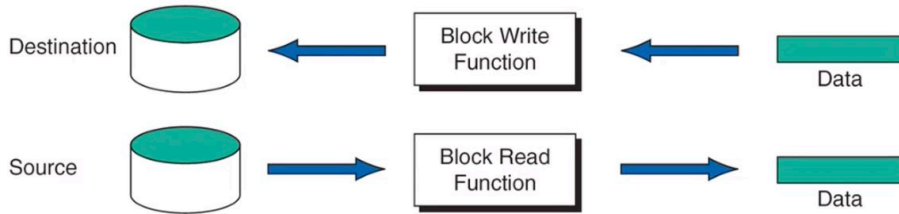


FIGURE 13-2 Block Input and Output

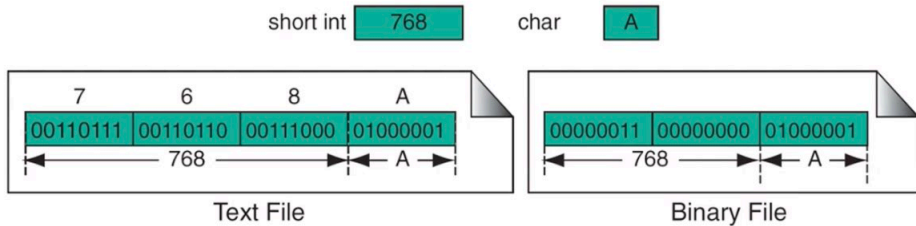


FIGURE 13-3 Binary and Text Files

Note

Text files store data as a sequence of characters;
binary files store data as they are stored in primary
memory.

Mode	r	w	a	r+	w+	a+
Open State	read	write	write	read	write	write
Read Allowed	yes	no	no	yes	yes	yes
Write Allowed	no	yes	yes	yes	yes	yes
Append Allowed	no	no	yes	no	no	yes
File Must Exist	yes	no	no	yes	no	no
Contents of Existing File Lost	no	yes	no	no	yes	no

Table 13-1 File Modes

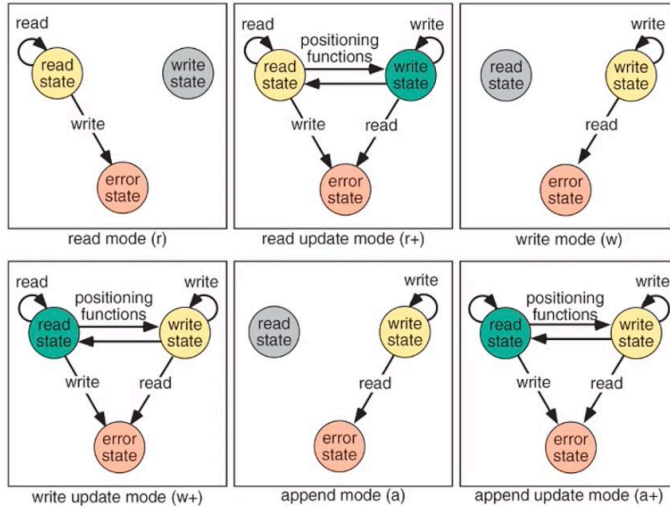


FIGURE 13-4 File States

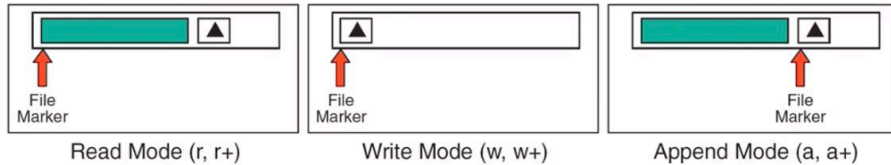


FIGURE 13-5 File-Opening Modes

13-2 Standard Library Functions for Files

C has eight categories of standard file library functions. We have already discussed before .

We discuss the other four categories, which are more related to binary files, in this section.

Topics discussed in this section:

Block Input/Output Functions

File Status Functions

Positioning Functions

System File Operations

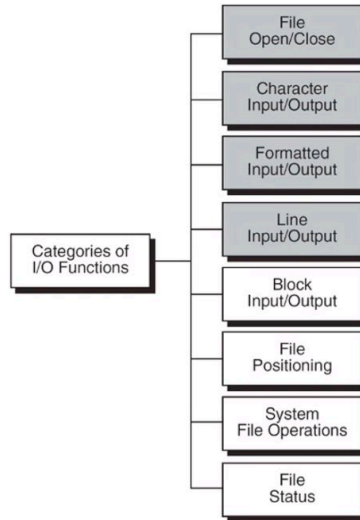


FIGURE 13-6 Types of Standard Input/Output Functions

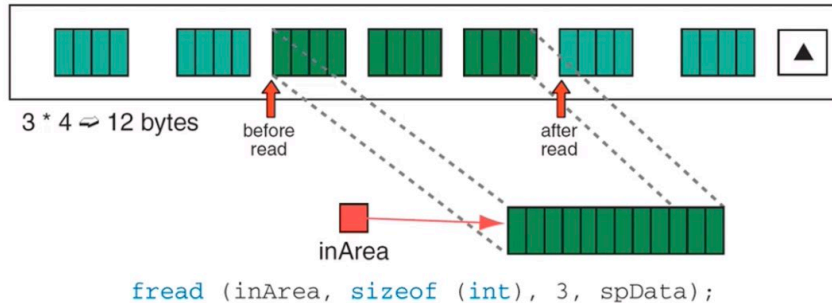


FIGURE 13-7 File Read Operation

```
1 // Read a file of integers, three integers at a time.
2 {
3     ...
4 // Local Declarations
5     FILE* spIntFile;
6     int    itemsRead;
7     int    intAry[3];
8
9 // Statements
10    spIntFile = fopen("int_file.dat", "rb");
11    ...
12    while ((itemsRead = fread(intAry,
13        sizeof(int), 3, spIntFile)) != 0)
14    {
15        // process array
16        ...
17    } // while
18    ...
19 } // block
```

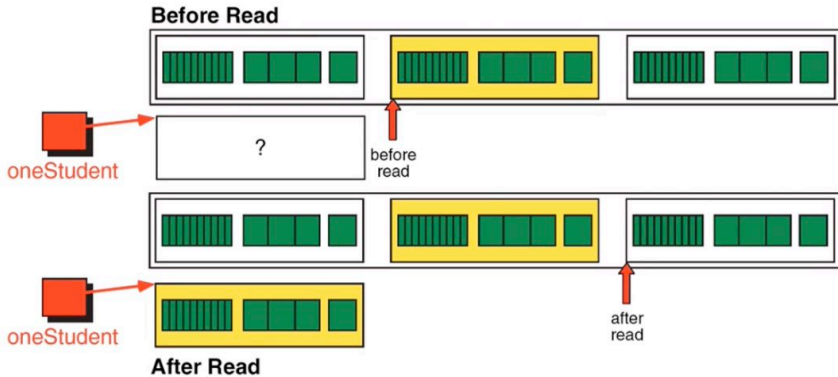


FIGURE 13-8 Reading a Structure

```
1  /* Reads one student's data from a file
2      Pre   spStuFile is opened for reading
3      Post  stu data structure filled
4      ioResults returned
5  */
6  int readStudent (STU* oneStudent, FILE* spStuFile)
7  {
8      // Local Declarations
9      int ioResults;
10
11     // Statements
12     ioResults = fread(oneStudent,
13                       sizeof(STU), 1, spStuFile);
14     return ioResults;
15 } // readStudent
```

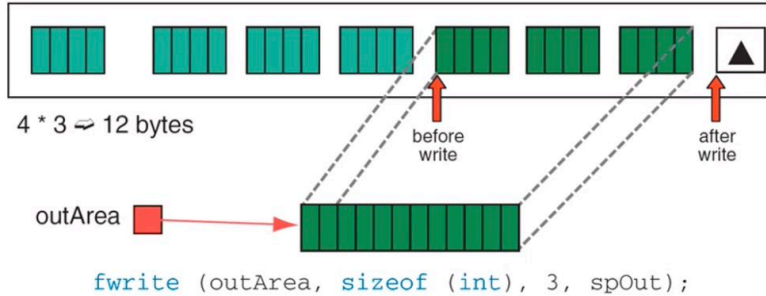
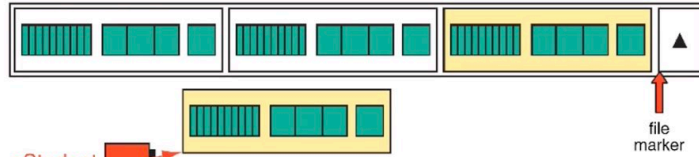
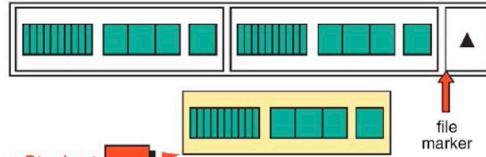


FIGURE 13-9 File Write Operation

Before Write



After Write

FIGURE 13-10 Writing a Structure

```
1  /* Writes one student's record to a binary file.
2      Pre  aStudent has been filled
3      spOut is open for writing
4      Post aStudent written to spOut
5  */
6  void writeStudent (STU* aStudent, FILE* spOut)
7
8  {
9      // Local Declarations
10     int ioResult;
11
12     // Statements
13     ioResult = fwrite(aStudent,
14                       sizeof(STU), 1, spOut);
15     if (ioResult != 1)
16     {
17         printf("\a Error writing student file \a\n");
18         exit (100);
19     } // if
20     return;
21 }
```

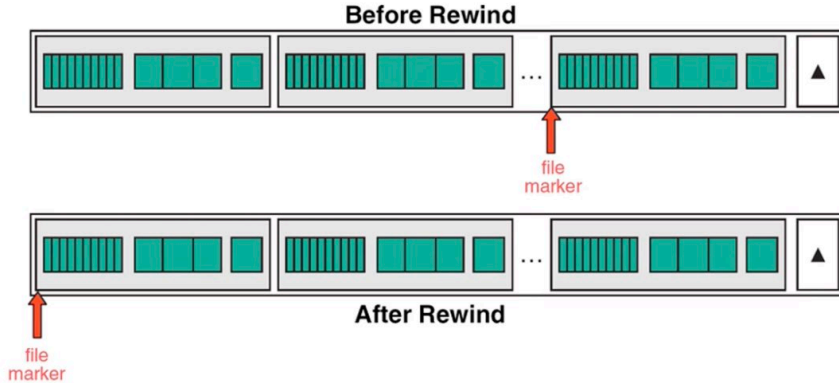


FIGURE 13-11 Rewind File

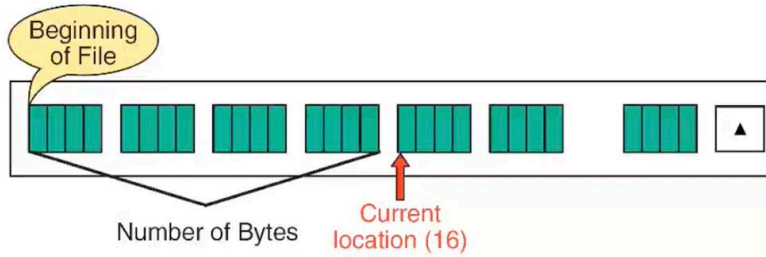


FIGURE 13-12 Current Location (*ftell*) Operation

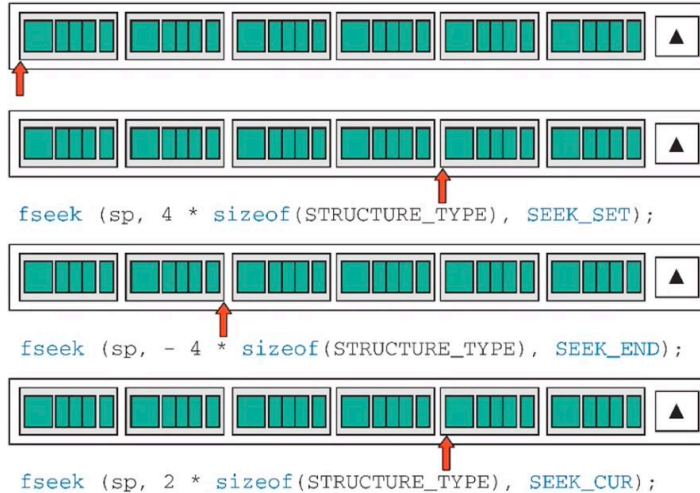


FIGURE 13-13 File Seek Operation

```
1  /* This program appends two binary files of integers.
2      Written by:
3      Date:
4  */
5  #include <stdio.h>
6  #include <stdlib.h>
7
8  int main (void)
9  {
10     // Local Declarations
11     FILE* sp1;
12     FILE* sp2;
13     int    data;
14     long   dataCount;
15     char   fileID[13];
16
```

```
17 // Statements
18 printf("This program appends two files.\n");
19 printf("Please enter file ID of the primary file: ");
20 scanf("%12s", fileID);
21 if (!(sp1 = fopen (fileID, "ab")))
22     printf("\aCan't open %s\n", fileID), exit (100);
23
24 if (!(dataCount = (ftell (sp1))))
25     printf("\a%s has no data\n", fileID), exit (101);
26 dataCount /= sizeof(int);
27
28 printf("Please enter file ID of the second file: ");
29 scanf("%12s", fileID);
30 if (!(sp2 = fopen (fileID, "rb")))
31     printf("\aCan't open %s\n", fileID), exit (110);
32
33 while (fread (&data, sizeof(int), 1, sp2) == 1)
34 {
35     fwrite (&data, sizeof(int), 1, sp1);
36     dataCount++;
37 } // while
```

```
38
39     if (! feof(sp2))
40         printf("\aRead Error. No output.\n"), exit (120);
41
42     fclose (sp1);
43     fclose (sp2);
44
45     printf("Append complete: %ld records in file\n",
46           dataCount);
47     return 0;
48 } // main
```